

One Person Lab

One Person Lab 白皮书

让复杂知识工作从一次问答走向持续交付

OPL 的核心承诺，是让论文、基金、汇报、专利等复杂成果不再停留在一次问答，而是能被持续推进、审阅、修订和交付。

OPL Framework / One Person Lab App / Foundry Agents

2026-06-08

Public whitepaper

目录

1	定位摘要	2
2	为什么要处理这件事	2
3	OPL 的答案：让 AI 像专业团队一样推进工作	2
4	这套设计从哪里来	3
5	认知计算：为什么不是固定流程	3
6	OPL 的三层产品模型	3
6.1	OPL Framework	3
6.2	One Person Lab App	4
6.3	Foundry Agents	4
7	设计哲学：让 AI 做专家工作，让系统守住边界	4
7.1	AI-first	4
7.2	Executor-first	4
7.3	Stage-led	5
7.4	Contract-light	5
7.5	Owner boundary	5
7.6	Progress-first	5
8	九个品牌模块：从上到下的设计骨架	5
8.1	OPL Charter	5
8.2	OPL Atlas	6
8.3	OPL Workspace	6
8.4	OPL Stagecraft	6
8.5	OPL Runway	6
8.6	OPL Vault	6
8.7	OPL Console	6
8.8	OPL Foundry Lab	7
8.9	OPL Connect	7
9	Foundry Agents：同一骨架，不同领域	7
9.1	MedAutoScience	7
9.2	MedAutoGrant	7
9.3	RedCube AI	7
9.4	OPL Meta Agent	8
9.5	未来 Foundries	8
10	为什么 OPL 能稳定运行	8
11	用户会如何感知 OPL	8
12	这份白皮书如何维护	9
13	参考与编制来源	9

适用对象：希望理解 OPL 品牌、产品定位、设计逻辑和 Foundry Agents 运行方式的用户、合作者、早期采用者和技术决策者。

核心判断：OPL 的核心承诺，是让论文、基金、汇报、专利等复杂成果不再停留在一次问答，而是能被持续推进、审阅、修订和交付。

1 定位摘要

- OPL 处理的问题不是一次回答是否足够聪明，而是复杂知识工作能不能持续推进到交付。
- OPL 不是一个聊天入口集合，而是面向论文、基金、汇报、专利、答辩、审稿等正式知识工作的智能体运行框架。
- One Person Lab App 是普通用户的工作台，负责把 framework 和 Foundry Agents 变成可见、可操作、可检查的产品体验。
- Foundry Agents 是运行在 OPL 上的专业智能体，例如 Research、Grant、Presentation 和未来的 IP、Award、Thesis、Review。
- OPL 的设计从用户要交付的成果反推到底层合同：用户要的是可靠推进，系统就必须提供阶段、权限、证据、文件、恢复和交接。

2 为什么要处理这件事

AI 已经很擅长回答一个问题、生成一段代码或润色一份材料。但真正让用户焦虑的，往往不是某一次回答好不好，而是一项复杂工作能不能真的走到交付。

一个人做研究、写基金、准备汇报、整理专利或推进答辩材料时，难点通常不是缺少一个聪明助手。真正困难的是：材料很多，工作跨越很多天甚至很多周，中间会反复修改、审阅、补证据、换方向、重开上下文。做了很多轮之后，用户经常不知道现在到底推进到哪一步，哪些文件是最新版，哪些判断已经被审阅，哪些问题还卡着。

这类任务也很难用普通项目管理工具解决。项目管理能记录任务名和状态，却不理解论文、基金、视觉交付或审稿的专业内容；普通自动化能执行固定步骤，却很难在阶段内完成理解、比较、创作、审阅和修订；普通 AI 对话能生成内容，却很难天然留下可恢复的阶段、证据、文件、owner 和交付边界。

所以 OPL 要处理的不是再做一个 AI 助手，而是给高价值知识工作做一个新的操作系统：让一个人也能拥有类似专业团队的持续推进能力，让 AI 不只是回答当下问题，而是把复杂成果一步步推到可以检查、可以修改、可以交付。

- 做了很多轮之后，当前到底推进到了哪一步？
- 中间用了哪些材料、改了哪些文件、留下了哪些证据？
- 准备、执行、审核、修订和交付能不能各有清楚边界？
- 人离开电脑后，任务能不能继续跑，回来时直接看到进展、阻塞和下一步？
- 多个专业 Agent 能不能共用一套运行、文件、进度和交付体系？

3 OPL 的答案：让 AI 像专业团队一样推进工作

One Person Lab 正是围绕这些问题设计的。它把复杂知识工作拆成一个个能推进的阶段：准备材料、执行创作、质量审核、修订完善、交付收口。

每个阶段都围绕一个真实成果增量工作。AI 可以在同一阶段里整理资料、提出候选方案、比较取舍、调用工具、接受审阅并继续修订。用户不需要盯着每个底层日志，但能够看到进度、文件、证据、阻塞和下一步都被清楚保留下来。

这就是 OPL 和普通对话工具最关键的差别。普通 AI 工具解决的是这一问怎么答；OPL 解决的是这项复杂工作怎么一步步做到能交付。它不要求用户相信某一次聊天记录，而是让用户能相信阶段、产物、回执、owner 和工作空间。

在这个模型里，AI 不被降级为表单填充器或固定脚本执行器。它仍然可以理解材料、比较方案、创造内容、接受反馈、再修订。系统负责让这些开放式工作有边界、有记录、有可恢复的落点。

- 用户看到的是任务进展和交付物，不是零散日志。
- Agent 接到的是阶段目标和可用能力，不是被硬编码的机械脚本。
- 系统保存的是证据、回执、阻塞和恢复点，不是无法复盘的聊天片段。

4 这套设计从哪里来

OPL 不是先有九个模块再去寻找用途。它的设计来源于长周期知识工作里的真实摩擦：一次性回答不能等于持续交付，自动化脚本不能等于专家判断，界面可见不能等于质量完成。

第一层来源，是 OPL 自己要服务的任务：论文、基金、汇报、专利、报奖、学位论文、审稿和修回。这些任务都有明确交付物，也都有大量中间判断。它们要求 AI 能做专业工作，同时要求系统能说清楚当前阶段、材料来源、文件位置、质量门、阻塞原因和下一步。

第二层来源，是 MAS、MAG、RCA 等 Foundry Agents 的真实实践。医学研究需要 evidence package、manuscript、publication quality gate；基金写作需要方向判断、申请书结构、模拟评审和修订；视觉交付需要 storyboard、render、review 和 export。不同领域的内容不同，但它们都需要同一类运行外围：stage、workspace、receipt、blocker、artifact lineage、projection 和 recovery。

第三层来源，是成熟工程系统的分层经验。OPL 借鉴的是 catalog、durable execution、operator pattern、artifact lineage、machine-readable descriptor 和 ADR 这样的治理思想，但不把外部系统变成第二 truth source。OPL 吸收的是如何组织复杂系统的原则，再把它落到 One Person Lab 自己的用户路径、专业 agent 和交付边界里。

- 从用户痛点来：复杂成果需要持续推进，不只是一次回答。
- 从领域实践来：MAS、MAG、RCA 证明不同专业 agent 需要共享运行骨架。
- 从工程经验来：可发现、可恢复、可审计、可派生的系统才容易长期维护。

5 认知计算：为什么不是固定流程

普通自动化流程擅长处理固定步骤：先做 A，再做 B，最后输出 C。复杂知识工作需要更强的阶段内判断：写一篇文章、改一个基金本子、做一套正式汇报，过程中经常需要反复判断、比较、推翻、重写和审阅。

OPL 把这种阶段内的开放式专家工作称为认知计算：AI 在一个可观察、可接力的阶段里完成理解、比较、创作、审阅和修订。系统不替 AI 决定每一步怎么想，也不把工具调用顺序写死；系统负责提供目标、材料、工具边界、权限、质量门和交接要求。

这样设计的好处是，AI executor 越强，OPL 的阶段能力就越强。模型升级、prompt 改进、skill 改进、domain knowledge 改进，都能直接提升阶段内的真实工作能力；同时，OPL 的 stage、receipt、workspace、projection 和 recovery 又能保证这些能力不会变成不可追踪的一次性对话。

用户最终感受到的不是这里有很多高级概念，而是这项工作真的在往前走。下一版文件形成了，证据能解释，审阅有边界，阻塞能被看见，交接能继续。

- OPL 不把专家工作压成固定脚本。
- OPL 让 AI 在 stage 内有足够空间完成真实判断。
- OPL 用系统边界保证开放式工作可见、可复核、可恢复。

6 OPL 的三层产品模型

为了让小白用户能直接使用，同时让专业 agent 能持续进化，OPL 对外呈现为一个品牌，对内保持三层清楚分工：Framework、App、Foundry Agents。

默认运行链路

```
Human / App / CLI
  -> Codex-default executor
    -> explicit OPL activation
      -> provider-backed stage runtime
        -> selected Foundry Agent
          -> domain-owned truth / quality gate / deliverables
```

6.1 OPL Framework

运行框架

Framework 是让复杂工作能跑起来并持续跑下去的底座。它持有 Codex-default session runtime、显式 activation layer、stage control plane、typed queue、provider-backed runtime、shared contracts、恢复和审计 surface。它负责让任务可以被启动、恢复、投影和收口。

- 默认第一公民 executor 是 Codex CLI。
- Temporal-backed provider 是 production online runtime 的必需 substrate。
- Framework 不持有 domain truth、quality verdict 或 artifact authority。

6.2 One Person Lab App

用户工作台

App 是普通用户使用 OPL 的产品入口。它把复杂 runtime 变成用户能看懂的工作台：选择任务、查看进度、打开文件、处理阻塞、接收更新。它消费 Framework 和 domain-owned projection，把任务、阶段、阻塞、交付物、source refs 和 next action 呈现为可操作界面。

- App 固定 Codex CLI concrete executor 的普通用户路径。
- App 负责 GUI product truth、release gate、用户教程和 active shell validation。
- App 不成为第二 runtime，也不替 domain agent 声明质量完成。

6.3 Foundry Agents

领域智能体

Foundry Agents 是 MAS、MAG、RCA、OMA 以及未来 Patent、Award、Thesis、Review 等领域智能体。它们像专业团队里的不同角色：研究、基金、视觉交付、智能体构建各自有材料理解、审阅标准和交付边界。它们以统一 OPL 结构运行，但保留自己的领域判断、质量 gate、artifact authority 和 direct skill path。

- 同一生命周期：材料进入、stage 执行、质量门、owner receipt、交付物 handoff。
- 不同领域保留不同输入、输出、知识、rubric 和 authority function。
- OPL 只投影 opaque refs，不读取或改写领域 body。

7 设计哲学：让 AI 做专家工作，让系统守住边界

OPL 最重要的设计逻辑，是把 AI 的自由度和系统的纪律同时保留下来。它不是用更复杂的规则替代 AI，而是把 AI 的开放式能力放进可审计、可恢复、可交接的边界内。

7.1 AI-first

能力来自更强执行者

复杂知识工作应该交给 AI executor 做开放式判断、写作、评审和修订。代码不把这些专家行为写死，只提供目标、上下文、权限、工具和质量边界。这就是为什么 OPL 能跟随更强模型、更好 prompt、更好 skill 一起变强。

- 模型升级、prompt 改进、skill 改进会直接提升系统能力。
- 合同只定义下限，不替代专家判断。

7.2 Executor-first

stage 内最小执行单位是 Agent executor

OPL 不是把任务拆成大量细碎函数，而是让一个被选中的 executor 在一个 stage 内完成接近人类专家的真实工作包。用户需要的是成果推进，不是看见系统把一项专业工作切碎成几十个机械动作。

- 默认 executor 是 Codex CLI。
- 非默认 executor 只通过显式 adapter 接入，并必须有 receipt、audit 和 fail-closed 语义。

7.3 Stage-led

大型任务按阶段推进

论文、基金、视觉交付和答辩材料都有自然阶段。OPL 把 stage 作为可观察、可恢复、可审计的工作单元，而不是把知识工作压成固定脚本。stage 让 AI 有空间完成专家工作，也让用户能看到工作推进到了哪里。

- 每个 stage 有目标、输入、输出、知识、工具边界和质量门。
- stage closeout 必须回到 owner receipt、typed blocker、human gate 或 route-back。

7.4 Contract-light

合同只守安全和可恢复下限

合同负责 owner、权限、safe action、receipt、blocker、audit、recovery 和 projection。它不声明论文质量、基金可中、视觉作品通过，也不把 readiness 当成 domain verdict。这样做避免系统用机械信号冒充专家质量判断。

- 机械检查只能定位缺口。
- 质量裁决必须来自独立 reviewer、domain quality gate 或 owner receipt。

7.5 Owner boundary

每个判断都有责任方

Framework、App 和 Foundry Agents 各有自己的 authority。OPL 管通用运行外围，domain agent 管领域 truth，App 管用户界面和 release evidence。清楚 owner boundary 是长期可维护的前提。

- 不制造第二真相源。
- 不让 UI 或 runtime 反向定义领域质量。

7.6 Progress-first

用户需要看见下一步

OPL 默认把 owner delta 放在最前面：当前等待谁、需要什么交付 delta、缺什么 receipt 或 blocker。用户看到的是可推进状态，而不是被底层计数淹没。因为用户关心的是工作怎么继续，不是系统内部有多少事件。

- 进度来自产物、证据、决策和交接。
- 诊断细节只在 drilldown 中展开。

8 九个品牌模块：从上到下的设计骨架

九个模块不是营销名称，而是 OPL Framework 内部能力的 bounded context。它们解释了 OPL 为什么能从顶层理念一路落到真实运行：先有语言和边界，再有目录和工作空间，再有阶段设计、长跑执行、证据回执、用户控制台、智能体工坊和外部连接。

8.1 OPL Charter

语言和边界

Charter 固定 OPL 的顶层定位、命名、产品层级、ADR/RFC 和品牌组合治理。它回答什么叫 OPL、为什么存在、什么属于 Framework、什么属于 App、什么属于 Foundry Agent。没有 Charter，系统很容易变成旧路线、新路线和临时说法的混合体。

- 让整个系统拥有统一语言。
- 防止旧路线和新路线混在同一叙事里。

8.2 OPL Atlas

可发现目录

Atlas 管理 agents、capabilities、surfaces、owners、dependencies 和 lifecycle catalog。用户只想知道我能让 OPL 做什么，系统则需要知道能力来自哪里、谁负责、如何调用、当前生命周期是什么。Atlas 把这些信息放进可发现目录。

- 负责 discoverability，不执行动作。
- 把 domain descriptor、module registry 和 public surface 放进统一目录。

8.3 OPL Workspace

用户和 Agent 共同检查的项目空间

Workspace 把材料、共享资源、stage outputs、handoff 和文件生命周期组织成可检查结构。复杂任务最终必须落到文件、材料和交付物上；用户能在项目空间里看到当前产物和交付物，而不必钻进 runtime backing directory。

- 默认模型是 Workspace Group -> Project Unit -> Stage Artifact Unit。
- 真实完成必须绑定 manifest、receipt 和 current pointer。

8.4 OPL Stagecraft

stage 内认知计算设计

Stagecraft 负责 stage 设计、prompt、skills、tool affordance、knowledge、rubric 和 independent quality gate。它回答这个阶段要让 AI 怎样像专家一样工作，并在保留开放式专家空间的同时，让输入、输出和边界可审计。

- 工具目录是 affordance catalog，不是硬编码流程。
- 复杂质量判断必须是独立 stage 或 quality gate。

8.5 OPL Runway

可恢复的长跑执行

Runway 负责 durable execution、typed queue、attempt、lease、retry/dead-letter、wakeup 和 human gate。它回答这项工作怎么持续跑下去。一个长期任务不应该因为进程退出、上下文切换或用户离开电脑而失去运行线索。

- 承接 provider-backed stage runtime。
- 不创建 domain verdict，不替代 executor 的专家工作。

8.6 OPL Vault

证据、回执和 lineage

Vault 保存 refs-only evidence、receipt refs、typed blocker refs、artifact lineage、restore proof 和 provenance。它回答为什么我们相信当前状态。任务为什么可以继续、为什么被阻塞、如何恢复，都必须有可追踪依据。

- 保存 refs，不保存 domain memory 或 artifact body。
- 让系统可审计，但不篡改领域 truth。

8.7 OPL Console

用户和 operator 的可见控制台

Console 把 readiness、current owner、next action、阻塞、产物和 drilldown 投影到 App/operator 工作台。它回答用户现在应该看什么、做什么。用户需要知道任务在哪里，而不是读底层 ledger。

- 默认 owner-delta-first。
- 只消费 projection，不成为第二 runtime。

8.8 OPL Foundry Lab

智能体创建与机制改进

Foundry Lab 支撑新 agent 创建、测试接管、mechanism improvement、canary、promotion 和 rollback。它回答 OPL 怎样持续变强。运行证据不会只停留在报告里，而会转成可执行改进任务。

- 用于改进 agent 机制。
- 不接管 MAS、MAG、RCA 或未来 agent 的 domain authority。

8.9 OPL Connect

外部调用和分发连接

Connect 把同一合同派生到 CLI、MCP、OpenAI/AI SDK tools、Skill/plugin、module install、release/install 分发和 drift matrix。它回答同一套能力怎样被安装、发现、调用和分享。

- 让能力可以被安装、同步、发现和调用。
- 不重新解释 domain 语义。

9 Foundry Agents: 同一骨架，不同领域

OPL 的品牌感很大一部分来自 Foundry Agents。用户真正需要的不是一个万能但模糊的助手，而是能进入具体专业场景、理解材料、形成成果、接受审阅的专业 agent。每个 agent 都面向一个高价值工作流，拥有自己的领域知识、质量门和交付权威，同时共享 OPL 的 stage-led 运行骨架。

9.1 MedAutoScience

Research Foundry

面向医学和科研论文工作流，从数据、文献、研究问题、分析、证据包到稿件和投稿包。研究工作特别需要证据、审阅、修订和投稿边界，所以 MAS 持有 research truth、publication quality gate、artifact authority 和 owner receipt。

- 用户感知：把研究项目持续推进到可审阅论文和投稿材料。
- OPL 只承载 stage attempt、queue、receipt projection 和 workspace/runtime 支撑。

9.2 MedAutoGrant

Grant Foundry

面向基金方向判断、申请书写作、作者侧模拟评审和修订。基金工作不是把论文改成申请书，而是重新组织问题、创新点、技术路线和评审说服力；MAG 复用研究证据和记忆，但保持 grant truth 与 fundability review 边界。

- 用户感知：从想法到申请书草稿、模拟评审包和修订计划。
- 质量判断回到 MAG 自己的 gate 和 owner receipt。

9.3 RedCube AI

Presentation Foundry

面向视觉交付、PPT、报告、storyboard、render、review 和 export。汇报材料需要叙事、结构、视觉和可展示文件同时成立；RCA 持有 visual truth、review/export verdict 和 artifact authority。

- 用户感知：把材料转成可展示的视觉交付物。
- OPL 负责 stage folder、manifest、receipt refs 和投影。

9.4 OPL Meta Agent

Foundry Lab managed module

面向 agent 创建、测试接管、机制改进和 target-agent work order。它帮助 OPL family 变得更会生产 agent、更会测试 agent、更会从失败中改进机制，但不持有任何领域 agent 的最终 truth。

- 用户感知：OPL 能持续变强、持续生成和修复自己的 agents。
- 边界：只生成改进 work order 或 typed blocker，不替 domain owner 签收。

9.5 未来 Foundries

IP / Award / Thesis / Review

IP、Award、Thesis、Review 等工作流会沿用同一骨架：材料进入、domain pack 解释、stage-led 执行、独立质量门、owner receipt 或 typed blocker、交付物 handoff、OPL refs-only projection。它们不会被硬塞进现有 agent，而是在对应 domain boundary 成熟后成为新的 Foundry。

- 每个新 agent 都必须有自己的 domain truth 和质量边界。
- 共享 OPL runtime，不复制私有通用 runtime。

10 为什么 OPL 能稳定运行

OPL 的稳定感不是来自把所有事情写成固定流程，而是来自把开放式智能工作放进一组可靠边界里。它让 AI 像专家一样工作，也让系统像专业组织一样留痕、交接和恢复。

第一，任务以 stage 为单位推进。一个 stage 足够大，可以让 AI executor 完成真实工作包；同时也足够清楚，可以定义输入、输出、权限、quality gate 和 closeout 形态。

第二，owner boundary 清楚。Framework 只持有 framework truth；App 只持有产品和展示 truth；Foundry Agent 持有 domain truth、quality verdict 和 artifact authority。这样每个判断都有归属，系统不会因为投影可见就误报完成。

第三，证据是 refs-only 的。OPL 不复制领域正文，不用 UI 或 ledger 伪造质量裁决。它记录 receipt、typed blocker、lineage、current pointer 和 restore proof，让任务可以被审计和恢复。

第四，普通用户默认看到 next action。OPL 不把用户困在底层事件、trace、计数和 debugging lane 里，而是先回答当前等待谁、缺什么、下一步能做什么。

第五，系统从单一来源派生多个入口。CLI、App、Skill、MCP、AI SDK tool、plugin 和 release/install 分发都围绕同一 contract surface 生成，避免每个入口各说各话。

第六，设计始终从用户要交付的成果反推。只要某个 surface 不能帮助用户看清进度、产物、证据、阻塞、owner 或下一步，它就不应该成为默认入口。这样的减法治理，保证 OPL 不会变成堆满诊断细节的工具箱。

- 这就是 OPL 的核心体验：AI 有足够自由度完成专家工作，系统有足够纪律保证工作可见、可复核、可恢复、可交付。
- 用户不需要相信一段聊天记录。用户可以相信阶段、产物、回执、owner 和工作空间。

11 用户会如何感知 OPL

好的智能体系统不应该让用户感到自己在调试工具。它应该让用户觉得自己在管理一个可以持续推进工作的团队。

在 App 中，用户选择的是任务和 Foundry Agent，而不是 backend、adapter 或 executor 细节。普通用户路径固定为 Codex CLI executor，复杂 runtime 和 provider 状态折叠到 readiness、current owner、next action 和 drilldown。

在工作空间中，用户看到材料、stage outputs、交付物和当前指针。运行状态可以被解释，但不会取代项目文件本身。真正的交付物仍落在用户可以检查、复制、分享和归档的位置。

在长期任务中，用户看到每个阶段的进度、阻塞和交接。任务可能需要人类批准、补材料、接受 route-back 或等待 domain owner receipt，但这些状态都会显式出现。

对小白用户来说，最重要的感受应该是：OPL 不是要求你理解一套复杂工程名词，而是替你把复杂工作组织起来。你只需要从任务入口进入，看当前阶段、打开产物、处理必要的阻塞，并在关键节点做判断。

- OPL 把复杂性收进系统，把可行动信息呈现给用户。
- OPL 把 AI 的创造力留在 stage 内，把责任和证据留在系统边界上。
- OPL 把一次性助手升级成可以持续运行的实验室伙伴。

12 这份白皮书如何维护

白皮书本身也遵循 OPL 的维护哲学：单一内容源、可复现生成、生成后验证。

内容源是 `docs/public/whitepaper/opl-whitepaper.source.json`。Markdown、PDF 和 verification JSON 都从这份 JSON 派生。这样做可以避免同一份白皮书在网页、PDF 和仓库文本中出现多套不一致的正文。

PDF 使用 Pandoc + XeLaTeX 生成，字体采用本机可用的 Noto Sans CJK SC。生成脚本会渲染 PDF 页面、抽取文本、检查关键内容、记录页数和工具链路径，方便后续维护者确认产物仍可分享。

外部白皮书风格参考了经典项目的做法：白皮书应先建立定位和信念，再解释系统模型；同时应保留当前性边界，不把历史或愿景写成未经验证的 production-ready 声明。

OPL 自己的 README 是这份白皮书的主要语气来源：先提出用户为什么需要 One Person Lab，再解释阶段式推进、专业 agent、进度证据和产品分层。白皮书比 README 更完整地交代设计来源和九模块结构，但重心仍然是让用户明白 OPL 为什么存在、为什么必须这样设计。

- 更新正文：改 JSON。
- 重新生成：运行 `npm run docs:whitepaper`。
- 验证分享版：检查 PDF 渲染页和 verification JSON。

13 参考与编制来源

- [Ethereum Whitepaper](#)：参考其白皮书页面对愿景、系统模型和当前性说明的组织方式。
- [Bitcoin whitepaper PDF](#)：参考其短而正式的技术宣言式结构。
- [Pandoc User Guide](#)：本仓采用 Pandoc + XeLaTeX 生成 PDF。
- [Typst PDF documentation](#)：作为后续可能迁移到更现代 PDF 工具链的参考。